

Modeling Photosynthesis Board Game with Linear Programming

*Note: Photosynthesis is a trademark of Blue Orange Games

Nicholas Smith
CS 412
Winter 2022

Abstract—A tutorial for representing the game of Photosynthesis as a linear programming problem to find optimal strategies for a single turn of play. A discussion of the decision variables, how to construct the constraints needed, and options for modeling the utility of each game action for maximization.

Index Terms—Photosynthesis, Linear Programming, Optimal Strategy

I. INTRODUCTION

Linear programming is a useful and powerful way of model for optimal solutions to many real-world problems. It has a wide range of applications from network flow and transportation problems to game theory to data science. The strategy is simple, any problem that can be represented with a series of linear constraints and a linear objective function to be maximized or minimized can be formulated as a linear programming problem. The intent behind this project and paper, then, was to broaden the applications of linear programming to find optimal strategies for the Photosynthesis board game. The method is not simple, but we show how a model can be built and constraints set up to accomplish this.

A. The Game of Photosynthesis

1) *Basic Rules and Play:* Beginning with a brief introduction to the game, Photosynthesis is played on a hexagonal grid, as shown in Figure 1. It can have between two and four players. Each player has a different species of tree and begins the game by placing two small trees of their color on the board. They also have a handful of other trees and seeds in their inventory which can be placed down later as well as a store from which they can purchase additional trees for their inventory.

The currency for the game is sun points, which are gained at the beginning of each round as the sun rotates to the next corner of the board, however, the number of points earned for each tree depends on the size of tree. Large trees generate 3 sun points for their player when the sun rotates, medium trees generate 2, and small trees 1. Seeds produce no sun points but are needed to grow additional trees. The board has six corners and the sun rotates around the board a total of three times, making for a total of 18 rounds during which players take their turns.

Players place trees onto the board during their turn by planting new seeds onto empty board spaces or by growing



Fig. 1. Photosynthesis Game Board and pieces

trees from smaller trees and seeds they already have on the board. Small trees can be grown from seeds using 1 sun point on the condition that a player has both a seed on the board and a small tree in their inventory. Medium trees can be grown from small trees for 2 sun points with the same conditions, and large trees can be grown from medium trees for 3 sun points. To grow a tree or seed, the larger tree from the player's inventory is placed onto the board space of the smaller tree it is being grown from and the smaller tree is then removed and placed back into the player's store. If there is no room in the store, the smaller tree or seed is simply discarded. Seeds can be placed onto empty board spaces for 1 sun point, however, they must also be placed within a certain distance of the player's other trees. The number tiles away they can be placed from a tree is proportional to the size of the parent tree in consideration.

Similar to growing a tree, once a tree has reached its full size it can be harvested for points if the player so chooses. Harvesting costs 4 sun points, but the points gained from it are different from the sun points used to make purchases and are what players actually need to win the game. There is a trade-off and balance, then, between keeping large trees on the board to generate sun and harvesting them to earn actual points for the game, and this is something players must take into consideration when choosing a strategy.

With these possible actions in mind, how the game works is players take turns on each round using their sun points to make purchases, plant trees, and try to grow their forest generally. There is no limit to the number of actions a player can take so long as they do not run out of their sun points, but the hope is that if they can plant and grow their trees now in an efficient and optimal manner, they can generate more suns and be able to grow and harvest more trees the next round, the objective, of course, being to have earned the most points from harvesting by the end of the game.

2) *Other Rules and Considerations:* There are a few caveats to consider, of course, while players make decisions about where to place and grow trees. One is that not all trees are guaranteed to generate sun points for the player each round. Trees cast shadows on each other as the sun rotates around the board, and any tree that is caught in the shadow of another tree of equal or greater size, including those that belong to the same player, will not generate any suns at the beginning of the round.

Another rule to consider is that each tree can only perform one action per round. In one sense, this makes the game easier for us to model, as we shall see later, however, as far as game play is concerned, this means a tree cannot grow twice in one turn. It also means a tree cannot grow and then be harvested in the same turn, and it also means a tree cannot grow once and then have a seed planted from it in the same turn, or vice versa. Each of these items counts as an action, so there can only be one per tree per round.

Lastly, not all harvests are created equal. Different spaces on the board have different levels of fertility, ranging from 1 (being the least fertile) to 4 (being the most fertile) and increasing as one moves from the outside of the board toward the center, and the fertility of a space determines how many points will be gained for a player when a tree is harvested there. More fertile spaces yield more points on average, but are more scarce and can be blocked more easily. Less fertile spaces yield fewer points but they generally have more access to the sun and there are more of them.

With all this in mind, players must strategize where to place their trees, then such as to maximize their expected harvest point yield while not compromising the amount of suns they earn between rounds. We hope to construct a model that will incorporate each of these attributes and considerations of the game and yield a set of optimal decisions a player can make.

II. FORMULATING THE GAME AS A LINEAR PROGRAMMING PROBLEM

As can be seen, the rules of Photosynthesis are not trivial. There is a lot for players to consider on any given round and thus trying to find the optimal strategy for such a game is also not trivial. It is different from a zero-sum game where players choose their actions at the same time and where an optimal equilibrium of choice frequencies exists between the players. Instead it is a turn-based game with no hidden information; one player can change their strategy to match another player's at any time as the game plays out.

This makes something like defining an optimal strategy or set of moves for the entire game an impossible feat. There is no problem we can set up that will look at the board from round one and determine exactly which actions a player should take every round through to the end of the game. Even a seemingly intelligent strategy such as considering all possible game outcomes from the current board set up and choosing the set of moves that gives the highest chance of winning down the line is also infeasible due to the sheer number of possible games to consider. A simple, hypothetical game with only a single player, for example, even if the player had a mere 10 spaces on average onto which they could perform actions each turn, after 18 turns the number of possible games would be on the order of 10^{18} . Add into each round the possible actions for two or three players and different options for the order or buying or growing trees and that number becomes even more astronomically high. Searching such a space to find the best possible set of decisions for a given round, then, also becomes impossible.

With that in mind, the goal with this paper and project is to look at a different approach to determining the optimal set of moves on any given turn. Rather than search the entire set of games branching from a single decision in order to determine how good that decision is, if we can instead find a measure that will simply predict the value of a given decision, we can potentially set up a linear programming problem that will maximize the utility of our actions while staying within the constraints and rules of the game. Such an approach is not only much more manageable, but also gives us the chance to explore and find insight into which strategies and measures for utility are better than others.

A. Model Overview

We start building our model by considering what needs to be included in a linear programming problem. The ideal model would account for every rule and every possible decision a player can make, however, to do so requires that we reduce the entire game and set of rules into something that can be represented by a linear objective function and set of constraints. We considered a few options for doing such.

We considered a few options for doing such. One, for example, might be to create a set of simplified objective functions for modeling different components of the game and using them together to make decisions. We could create one to give us the optimal way to buy trees at a minimum cost (although, such a problem is trivial unless we account for replenishing the store because players have to buy the trees with the minimum cost first anyway); We could create another to give us the best way for a player to place their trees on the board once they had already bought them, in order to maximize the sun points earned on the next turn. This could be done, for example, by letting $\Delta x_j \in \{0, 1\}$ be a decision variable, where Δx_j tells a player how much to increase the size of tree j in order to create the highest yield of new suns given the limited number of suns available on the current round.

The problem with these and other approaches is that such sub-problems do not necessarily bleed one into the other to tell a player exactly which actions all across the board to choose on their turn. The above system for trying to maximize sun points, for example, would fail to account and allow for harvesting unless we could somehow tell it that harvesting a tree, buying it again, and placing it somewhere else on the board might be a way to yield more sun. But that would require somehow blending the sun-maximizing problem with the tree-buying problem aforementioned.

It is this desire to have one action flow into another that leads us to our next approach to solving the problem, which takes some inspiration from the commonly known Network Flow and Transportation problems. Such an approach, as we shall see, will give us exactly what we are looking for, a model that accounts for nearly all the rules and yields as its result the exact set of all the actions they should take given the current setup of the board.

1) *The Graph Approach:* The idea behind the graph approach to modeling this problem is that on each turn of the game a player has only a finite and discrete set of actions they can take on their turn, the actions as mentioned before, flow into each other, and in general, no action can be repeated twice. If a player has two medium trees in their inventory, for example, and one small tree on the board in space k_1 , they may choose to grow the small tree into a medium tree, moving the medium tree from their inventory to space k_1 , however, they can never move two medium trees from their inventory to space k_1 . They may, however, choose to move their second medium tree to a different space k_2 on the board that also has a small tree and this would be fine.

If we can determine all of these possible actions, then, which can be done easily by simply looking at the current setup of the board, and then model them as edges in a graph representing the flow of trees from the store to the board and back, this will give us the basis for setting up a problem with linear constraints which we can use to tell a player which actions to include in their choice for their turn. Our decision variables become our edges in our graph, which can either be included or not (set to 0 or 1), and our objective function becomes some measure for the utility of each action which we try to maximize. We can formulate the constraints of our problem, as mentioned, then, based on the rules of the game.

B. Building the Basic Model

In order to build up a linear programming representation of our game as described above we will first build up a model that includes all of the basic actions available to players on their turn and then we will branch off from there to ensure we are accounting for all of the rules of the game.

1) *Buying Trees:* The first action we will consider is that of buying a tree from the store. It may not be readily apparent how we can accomplish this when trees on different spaces in the store, even of the same size, have different costs associated with them that we have to distinguish. However, using a graph approach, we can deal with the discrete nature and cost of

each store space, by simply letting each space i in the current player's store be represented by a node in our graph s_i for $i = 1 \dots 13$.

As we label each space we will want the seed space on the lower left to be represented by $i = 1$, and then work our way from the bottom of the board to the top and from left to right until we reach $i = 13$ representing the space for a large tree with the greatest cost (see Figure 2). Doing this this will ensure that the spaces are labeled in order of increasing cost and that store spaces of the same tree size are labeled in consecutive order. This will become important later, however, for now note that we will also want to create four sets, $S_1 \dots S_4$ to distinguish which store spaces hold which size of tree, where $s_i \in S_u$ if s_i houses a tree of size u . Here and throughout this paper we will use $u = 1$ to represent the tree size of a seed and $u = 4$ will represent the tree size of a large tree. A tree size of 0 will represent an empty space on the board. Once we have created



Fig. 2. Example of a player's store supply board.

each node s_i and each set S_u , we are ready for the next step in our problem formulation, which is to create separate nodes representing the player's inventory supply of each tree size. If we let node t_j represent the player's inventory for trees of size j , then we can model the action of a player buying a tree from their store and moving it to their inventory by simply creating an edge s_{ij} from each $s_i \in S_{u=j}$ to each node t_j for $j = 1 \dots 4$. If we let the cost as_{ij} of s_{ij} be set to the cost in sun points of purchasing a tree from store space i , then taking edge s_{ij} in our solution at cost as_{ij} will represent the action of buying a tree from store space i for cost as_{ij} .

2) *Growing Trees:* In a similar manner to how we modeled buying trees, model the placing of trees and seeds onto the board is also fairly simple. We begin by creating a series of nodes x_k to represent each space on the game board and letting $\hat{x}_k$ represent the size of the tree on board space k , with the same size convention described in the previous section. With this setup, we can represent the action of placing a tree of size j onto the board by drawing an edge t_{jk} from each inventory

node t_j to each board space node x_k where it is valid for a tree of size j to be placed, that is to any node x_j where $\hat{x}_k = j - 1$. If we then let the cost at_{jk} of each edge t_{jk} be the cost of placing a tree of size j on the board, which is 3 for large trees, 2 for medium trees, and 1 for both small trees and seeds, then including an edge t_{jk} in our solution will adequately represent the action of growing a tree and we can continue with our problem formulation.

Figure 3 shows an example of what the described setup looks like for the two store nodes tied to large trees, assuming the player has two medium trees on spaces 6 and 32.

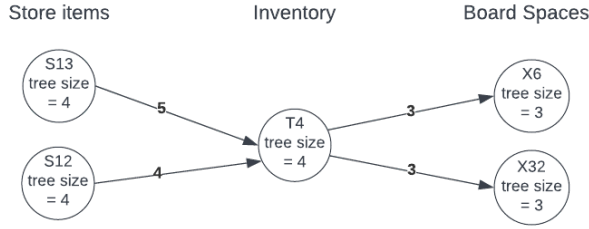


Fig. 3. Model of a few basic actions for a turn where the player has two large trees in their store and two medium trees in positions 6 and 32 on the board

3) *Harvesting*: The next item we need to consider, however, is harvesting. This was the function of the game that gave each of our previously discussed models trouble. We can address it here, however, by considering that in a lot of ways harvesting is like growing trees and as such can be modeled as such.

We accomplish this by creating a fifth inventory node t_5 that we can use to represent our supply of harvests for the round. No tree of size 5 actually exists, but we can think of a harvest being a tree of size 5 that a player always has a supply of whenever they have a large tree on the board. We can then create an edge t_{5k} with cost $ct_{5k} = 4$, the cost of harvesting, from node t_5 to any node x_k that has a large tree on it, or where $\hat{x}_k = 4$, and treat node t_5 just like all of our other inventory nodes. The only difference is it will have no edges coming into it from the store. Figure 4 Demonstrates what this would look like.

We could, of course, argue that after a harvest the board space k where the harvest is left empty and thus could have a seed planted to it from another tree. We could attempt to include this aspect of the game into our model by including additional edges from the seed inventory node t_1 to any board space node x_k where $\hat{x}_k = 4$ and try to include some constraints governing their use. Because this is an edge case, however, and not essential to the normal play of the game, for our purposes we will simply ignore this possibility for now and treat each board space, instead of each tree, as if it can only have one action performed on it per turn. This will help to keep our model simple while not drastically altering game play or breaking the rules.

4) *Constraints and Objective Function*: Having our basic model created an graph set up, we need to start considering again how to reduce our model to a linear programming problem. We start by considering again our objective function. Our goal is to predict which spaces on the board will be more likely to lead a player to win if they develop on them, however, we also now have which variables we are tied to and represent such actions, namely our t_{jk} variables. With this in mind, if we use c_k to represent the utility of developing on space k of the board, then we can start to create our objective function and linear program by representing it with the follow:

$$\text{Maximize } \sum_k c_k t_{jk}$$

Once we have this objective setup, however, we will also need to start considering the constraints we will need for our problem. We currently only have the basic setup of the game in our current model, however it will still be helpful to set up some these constraints before our model gets overly complicated. There are at least three we need here.

The first constraint we need is quite simple and has already been alluded to. We can recall that we are treating our graph edges s_{ij} and t_{jk} as our set of decision variables and that they will be either included in our decision set or not. This means our first constraint needs to be $s_{ij}, t_{jk} \in \{0, 1\}$.

This setup this setup is not 100% true to play, of course, as it will limit the number of items that can be bought from any store space to one per turn while in reality, a player can replenish an item in their store and buy from it as many times as the want to in their turn so long as they do not run out of sun points. This, however, is once again an edge case situation that does not happen often and is not critical to normal game play, so once again, to keep our model simple, we will also ignore this for now and be satisfied limiting each store space to be bought only once.

That does lead us to our second constraint, however, which is to state that the total cost of all the actions included in our solution cannot exceed the number of sun points our player has available at the start of their turn. Denoting this number as b_1 , we can enforce this rule by giving give following constraint:

$$\sum_{i,j} cs_{ij}s_{ij} + \sum_{j,k} ct_{jk}t_{jk} \leq b_1$$

That is, that the sum of all our decision variables, or the actions included in the solution, times their cost has to be less than or equal to b_1 .

Last, we need to create a way to limit the number of trees our algorithm attempts to place on the board to the number of trees the player actually has in their inventory. We can do this easy enough as well by letting t_j also represent the supply j in inventory node t_j at the start of a players turn and then then formulating a constraint on the tree supply supply going out of each node t_j by stating that the $\sum_k t_{jk} \leq t_j + \sum_i s_{ij}$. This means that the number of trees leaving an inventory node has to be less than or equal to the supply of trees already there

plus the number of trees coming into the that node from the store. Put in standard form we have the following:

$$\sum_k t_{jk} - \sum_i s_{ij} \leq t_j, \forall_j$$

Of course, with harvesting, no real supply of harvests available, however, we can generalize this constraint to include harvests by simply setting t_5 equal to the number of large trees the player has on their board at the start of the round as the number of harvests a player chooses to do in a turn can never exceed this number.

Figure 4 illustrates our updated model including both our harvest node and some example supplies for our inventory nodes.

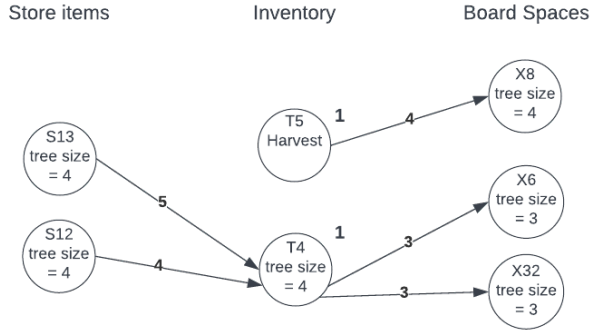


Fig. 4. Demonstration of how to model harvesting and accounting for the supplies t_j . Here we have constraints $t_{4,6} + t_{4,32} - s_{13,4} - s_{12,4} \leq 1$ and $t_{5,8} \leq 1$

C. Other Considerations

Of course, even with all of our basic problem set up taken care of and an idea of what our objective function will be, our model still isn't finished and we have still only scratched the surface as far as how much of the game we can include in it. This section explores some other key components and rules of the game and how we can formulate constraints to ensure these are met as well.

1) *Replenishing the Store:* The first additional issue we need to address is the ability for players to replenish the trees in their store, which we have brought up from time to time.

We can start to address this by assigning each of the store items s_i a supply value similar to the values for each of the player's inventory node. The difference here is that each store space is distinct and can only have one tree there at a time. This means we can set supply s_i to 1 if store space i has a tree in it at the start of the round and 0 if does not. However, we also need a constraint, then that says if a store item's supply is zero, the player cannot buy that item unless it is replenished. For this we will need to add a few more edges.

We can recall that trees get placed back in the store only when they are replaced on the board by bigger trees and that this action is always attempted, but only successful if the store

actually has an opening. One way we can model this action of attempting to put a tree back in the store is by creating an edge x_{ki} from any node x_k , where k is a space on the board that has a tree or seed on it of size u , to each empty store node s_i where $s_i \in S_u$. This is the same as creating a set of edges $\{x_{ki} \mid \dot{x}_k > 0, s_i = 0, s_i \in S_{u=\dot{x}_k}\}$.

We only need to draw edges to empty store nodes because our limit on the number of trees coming out of each node is 1. If a store node s_i has a supply of 1 and gets replenished, the new tree cannot be bought, only the one that was already there. Any edges going into such nodes are unnecessary, then, and because they will also conflict with how we set up our later constraints, we will simply exclude them.

The one exception to this may be if all $s_i \in S_u$ for some u are equal to 1. In this case we can create edges from all x_k with tree size u simply to $s_i \in S_u$ with the smallest cost. This will not do anything to affect our model as the constraint aforementioned only applies when some $s_i \in S_u$ is equal to 0 for some u . Trees of size u coming off the board in this case will simply accumulate and not be used. Keeping at least one edge coming into the store from each of the board spaces with trees will make defining some of our other constraints down the line more generalizable, however.

We can let the cost cx_{ki} for each edge c_{ki} equal 0 because putting a tree back into the store does not take any sun points. This will allow us to include such edges in our solution for free. This also means we do not need to include these edges in our sun point constraint, however, as they will not affect it if added to our solution.

An example of our model built up to this point can be seen in figure 5.

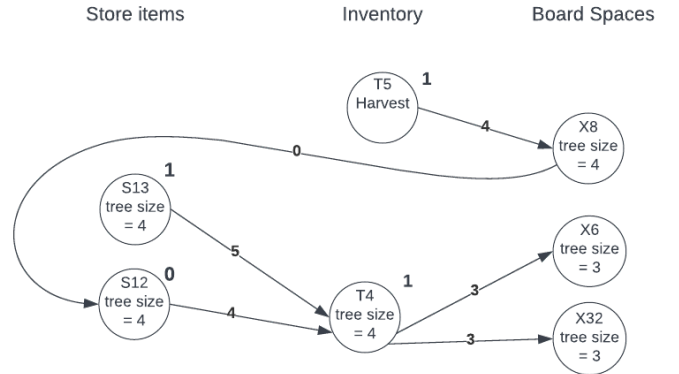


Fig. 5. Model of graph for replenishing the store

With these new edges, the constraints we need to add to our problem are as follows.

The first is similar to the constraint for our inventory supplies. That is that the number of trees being bought from a store space must be less than or equal to the supply there at the beginning of a player's turn plus any supply that gets added to it from replenishing. This is represented by

formulating a constraint for each of our store nodes such that $s_{ij} \leq s_i + \sum_k x_{ki}$. Put in standard form that equates to:

$$s_{ij} - \sum_k x_{ki} \leq s_i, \forall_i$$

We already have the constraint that $s_{ij} \in \{0, 1\}$, so we do not need to revisit that. However, we do also need to put some constraints on each edge x_{ki} . Specifically, we need to ensure that a store node is not replenished unless a board space with an equivalent tree size is grown somewhere. We can accomplish this by simply stating the following:

$$\sum_i x_{ki} = t_{jk} \quad \forall_k | \dot{x}_k > 0$$

This way any time an edge into a board space is chosen in the solution, or equals 1, an edge going out of it and back to a store space must also be chosen and set to 1.

2) The Order of Replenishing and Buying From the Store:

The two issues we do have to deal with, however, in attempting to include use of the store in our model, is considering the order in which trees must be bought from and replenished to the store.

For the first issue, when a player buys a tree of size u from their store, the rules and setup of the store board dictate that they must buy the tree from $s_i \in S_u$ that has the lowest cost. This rule is not too terribly difficult to deal with because by trying to maximize utility within a fixed amount of sun points, it is optimal to buy the trees of size u with the lowest possible cost anyway. Ideally, then, the algorithm used to solve our problem would give us this result without the need for additional constraints. However, it is also true that not all store spaces have different costs. Usually at least two spaces in the store for a given tree size share the same cost, the exception being for the largest trees. While we may argue that such store nodes are interchangeable, and it therefore does not matter which one the algorithm picks, if we want to strictly follow the rules of the game and sort out these ties, we have to devise a system for this.

This still does not necessarily require adding additional constraints, however. An easy way to solve this problem is to add a small ϵ_i coefficient for each s_{ij} variable in our objective function. So long as $\epsilon_i > \epsilon_{i+1} \forall_i$. Doing this, because we labeled our store nodes in order of increasing cost, will guarantee that the edge with the lowest i , or lowest cost will be picked over any other edge that had the same cost, so long as it has supply there.

The second rule we have to deal with is slightly more complicated and we will need to create additional constraints in order to solve. It is that when a tree of size u is replenished to the store, it must be replenished to the empty space $s_i \in S_u$ with the greatest cost. Referencing back to Figure 2, this is the open space highest on the board for tree size u , or the space, as we have it set up, corresponding to the highest i . Because of our labeling convention, however, and because we only created edges from board spaces to empty store nodes, this makes it somewhat easier for us to define a constraint

that will solve this. Considering we generally only have edges x_{ki} coming into empty store nodes, saying that store space s_i must be replenished before store space $s_{i-1} \forall_{s_i, s_{i-1} \in S_u} \forall u$ is the same as saying:

$$\sum_k x_{ki} \geq \sum_k x_{k(i-1)} \quad \forall_i | s_i=0, (s_i, s_{i-1}) \in S_u \quad \forall u$$

This way s_{i-1} can never be replenished before s_i if s_{i-1} and s_i are of the same tree size.

3) *Planting Seeds*: Lastly, we simply need to account for the fact that if a seed is placed on the board it must have come from a parent tree somewhere, and this counts as that tree's action for the turn.

We can do this in a similar fashion to the way we solved the store replenishing problem. That is, we create an edge λ_{cp} $\forall_{cp}$, where c is the index of a currently empty board space (the potential child) within the seed range of the tree on board space p (the potential parent). See Figure 5 for a visual.

We do this so that we can say planting a seed is an action, so we need to now add a constraint saying that the sum of all the actions going to any board space in a turn is less than or equal to 1. We did not need that constraint before because it was inherent in the fact that before we only had one edge going into each board space from whichever inventory could potentially place a tree or seed there, but now that we can have two or more edges going into each board space, we need to say that $\sum_k t_{jk} + \lambda_{ck} \leq 1, \forall_k$.

Similarly, we need a constraint saying that a seed had to have come from a parent if it was planted, so we state that $\sum_k \lambda_{kp} = t_{1k}$, or in other words, if a seed is moved from the seed inventory (which is t_1) to board space k , making $t_{1k} = 1$, then of all the edges going from space k to its potential parent trees, exactly one of them has to be equal to one. No seed can have more than one parent. And if $t_{1k} = 0$ and no seed was planted in space k , then all of the edges going to parent trees must be 0 as well, so that no trees have an action counted against them.

And with that we should have a functional set of constraints and decision variables to represent our entire problem and let us maximize what we want.

D. Our Final Linear Programming Problem

With all of our constraints set up we can now define our linear programming problem as follows:

$$\begin{aligned} & \text{maximize } \sum_i \epsilon_i s_{ij} + \sum_k c_k t_{jk} \\ & \text{subject to:} \\ & \sum_{i,j} c s_{ij} s_{ij} + \sum_{j,k} c t_{jk} t_{jk} \leq b_1 \\ & \sum_k t_{jk} - \sum_i s_{ij} \leq t_j \quad \forall_j \\ & s_{ij} - \sum_k x_{ki} \leq s_i \quad \forall_i \\ & \sum_i x_{ki} = t_{jk} \quad \forall_k | \dot{x}_k > 0 \\ & \sum_k x_{ki} \geq \sum_k x_{k(i-1)} \quad \forall_i | s_i=0, (s_i, s_{i-1}) \in S_u \quad \forall u \\ & \sum_j t_{jk} + \lambda_{ck} \leq 1 \quad \forall_k \\ & \sum_p \lambda_{kp} = t_{1k} \quad \forall_k | \dot{x}_k = 0 \end{aligned}$$

$$s_{ij}, t_{jk}, x_{ki}, \lambda_{cp} \in \{0, 1\}$$

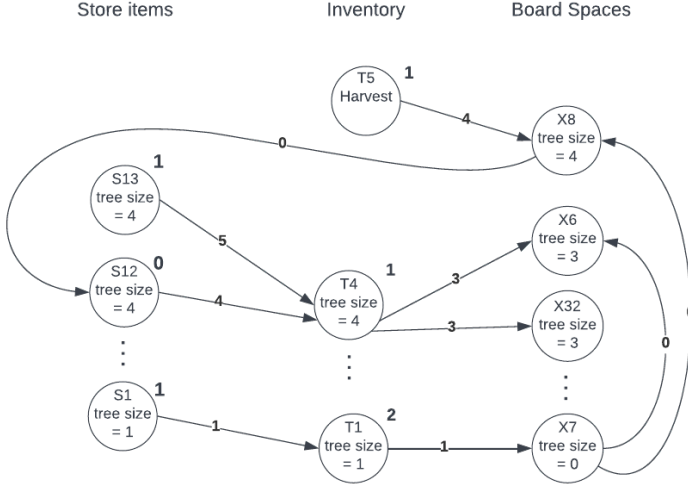


Fig. 6. Snippet of complete game model including seed to potential parent edges

E. Objective Function Considerations

With all of our constraints set up, it is time to come up with a definition for what exactly it is we are trying to optimize. We have mentioned before some ideas for what may be considered in defining the coefficients for our objective function, however in order to discuss them more in detail we may want to ask a few questions pertaining what might be important in predicting the value of a certain space on the board and which spaces we be improving. One question might be, for example, which spaces if we put a tree there are most likely to give the greatest returns in sun points? Which ones are least likely to be blocked by other trees? On which ones can we harvest for the most points?

Depending on our game strategy we could also ask other questions too, such as, which spaces if we improve them will minimize our opponent's sun points? Which spaces if we improve on them will prevent our opponents from harvesting for lots of points?

Both of these groups of items are valid to consider, however, we will focus merely on trying to maximize a player's own potential for points without worrying too much about what the other players are doing. The details we have chosen to consider are outlined below.

1) *The Size of the Tree already on the Space:* The first item we can consider is the size of the tree already in each space on the board. This is important because having a big tree for, say, the next three turns will potentially earn a player a lot more sun points than having only a small tree or a medium tree or a seed there. In addition, having bigger trees means a player is closer to harvesting and earning more points. They cannot win if they never push to get big trees on the board.

With that in mind, we want a space to show more potential, then, if it has a bigger tree there to begin with. We also might want that figure to be exacerbated more the bigger the tree is

because bigger trees cost more but tend to be worth it. We don't spaces that cost more to develop, then, discouraging our algorithm from choosing them.

One way to accomplish this is to create a measurement ρ_k that tells the progress, in terms of sun point, a tree in space k is. A player needs 1 sun point to plant a tree, 1 to grow it to a small tree, 2 to grow it to a medium tree, 3 to a big tree, and last but not least, 4 to harvest it. That is 11 points total. ρ_k , then could be defined as follows:

$$\rho_k = \begin{cases} 0 & \text{if space } k \text{ is empty} \\ 1/11 & \text{if space } k \text{ has a seed} \\ 2/11 & \text{if space } k \text{ has a small tree} \\ 4/11 & \text{if space } k \text{ has a medium tree} \\ 7/11 & \text{if space } k \text{ has a large tree} \end{cases}$$

2) *The Short-Term Potential for Sun Points:* The next natural factor to consider is the potential for a space to earn sun points within the next few rounds of game play. One good way to measure this was to ask the hypothetical question, if the whole board as it stands were to grow by one tree size between rounds for the next three rounds, how many of those rounds would board space k get hit with the sun. This, of course, is a number that would have to be calculated by simulating the next three sun rotations of the game. Once we do this, though, we can create a σ with σ_k being equal to the number of sun hits space k got. If a space got hit all three times, then that tells us there is a lot of potential for sun points there and we might take that into consideration in choosing to develop there.

With this measure, though, we do have to be somewhat careful. We want to devise a utility function that will give us a high utility if we should perform some action on a space and low if we should not. If a small or medium tree, or even a seed is on a space with high sun hits for the next three rounds, that tells us we may want to grow that tree so that every time it gets hit it will generate more sun points, so we want the utility to show as higher. If a large tree is on a space, however, leaving it on the space will generate three sun points every time it gets hit, while harvesting it will leave the space empty and generate none. We may want to show the utility of a space with a large tree on it to be lower, then, when there is a lot of potential for sun so that the algorithm does not tell us to harvest it. One way we can show this is by starting to form our c vector as follows, letting

$$c_k = \begin{cases} \sigma_k(1 - \rho_k) & \text{if } \dot{x}_k = 3 \\ \sigma_k \rho_k & \text{otherwise} \end{cases}$$

Where tree sizes are defined as follows

3) *The Long-Term Potential for Sun Points:* On the flip side, we also want to be able to tell something about how often a space may be blocked overall throughout the course of the game. This doesn't matter so much for large trees because when they are close to being blocked they can be harvested and moved somewhere else, but when considering something

like planting a seed, we want to try to pick spots that will not be blocked very often over spots that. Thus we want this aspect to weigh into our decision more if the tree on a space is smaller, and most if we are considering planting a tree there.

The blockability of a space is easy to measure, simply count up how many other tiles on the board could potentially have a tree there that could block it. What is nice about the photosynthesis board is that because it is a hexagon of circles, this number is the same for all circles on the same ring. Specifically, spaces in the outer ring can be blocked by 9 other spaces, spaces in the outer middle ring can be blocked by 12, other spaces, spaces in the inner middle ring can be blocked by 15 other spaces, and the space in the middle can be blocked by 18 other spaces. There are 37 total spaces on the board, so if we measure the blockability β_k of a space as the percentage of other spaces on the board that could potentially block it, then we get

$$\beta_k = \begin{cases} 9/36 & \text{if space } k \text{ is in the outer most ring} \\ 12/36 & \text{if space } k \text{ is in the second outer most ring} \\ 15/36 & \text{if space } k \text{ is in the second inner-most ring} \\ 18/36 & \text{if space } k \text{ is the center space} \end{cases}$$

4) *The Potential for Harvest Points*: Of course, the interesting thing about Photosynthesis is that, as we discussed earlier, directly tied to the blockability of a space is the fertility of a space. There are four different point piles for harvests in the game and the one a player picks from when he harvests a tree depends on the ring the tree was planted in. Points for harvesting a tree in the center of the board are much more than points earned for harvesting a tree in the outer ring. It would be good, then, to take this into account to when trying to make decisions about where to plant trees. If we let p_f for $f = 1 \dots 4$ be the average value of the remaining points in point pile f , 1 being the least fertile, 4 being the most fertile, then we can create a vector π representing the expected value of a harvest on each space such that

$$\pi_k = \begin{cases} p_1 & \text{if space } k \text{ is in the outer most ring} \\ p_2 & \text{if space } k \text{ is in the second outer most ring} \\ p_3 & \text{if space } k \text{ is in the second inner-most ring} \\ p_4 & \text{if space } k \text{ is the center space} \end{cases}$$

We can also update our c to reflect our desired changes. We want c to include $\pi_k(1 - \beta_k)$ for any k , meaning the potential for points on space k times 1 minus the blockability of space k long term, to tell us the kind of overall potential for a space. We also want this to weigh in more for small trees than for large trees like we said, which may be close to harvest anyway, and can't change their space at this point anyway, so we multiply that by $(1 - \rho_k)$.

We can now define c as

$$c_k = \begin{cases} \sigma_k(1 - \rho_k) + \pi_k(1 - \beta_k)(1 - \rho_k) & \text{if } \dot{x}_k = 4 \\ \sigma_k\rho_k + \pi_k(1 - \beta_k)(1 - \rho_k) & \text{otherwise} \end{cases}$$

5) *The Progress of the Game*: The last item we want to consider in determining which spaces on the board we should prioritize is going to be the progress of the game. Early in the game we want to focus more on growing up our small trees and harvesting suns, but later on in the game, especially the last few rounds, we want to really push for harvesting. So as the game goes on, I also want the value of places with large trees on them to go up so that the algorithm will push them to be harvest. A simple way to do this was, similar to measuring the progress of trees, measure the progress of the game as a percentage of the overall rounds completed. I also want this measure to be multiplied by the progress of the trees so that big trees will get more weight later in the game. If we let N equal the number of rounds in the game (typically 18) and n equal the turn number and then finish defining our objective function coefficients as:

$$c_k = \begin{cases} \sigma_k(1 - \rho_k) + \pi_k(1 - \beta_k)(1 - \rho_k) & \text{if } \dot{x}_k = 4 \\ \sigma_k\rho_k + \pi_k(1 - \beta_k)(1 - \rho_k) + n\rho_k/18 & \text{otherwise} \end{cases}$$

III. CONCLUSION

With the creation of our objective coefficient matrix, and our set of decision variables and linear constraints defined, we now have a complete and functional model for running Simplex to find optimal solutions for the game of Photosynthesis. We may, of course, wish to test our objective function and model before we claim it is the best way to go about playing the game. We could, for example, attach various weights w_v to each component in our definition of each c_k to define different strategies and run a bot that will play the game against itself with each of the different strategies to see which ones give better results. For now, however, we have the hard part completed, which is setting up the problem to begin with, and that gives us plenty a good start for being able to run different tests and do research with it in the future.